

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORY 6

Inside the xv6 File System

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3

INTRODUCTION 4

1 DIRECTORY REPRESENTATION AND FILE TYPES 5

1.1 Filesystem Type Enumeration 5

1.2 The Root Inode and Directory Self-Reference 5

1.3 Directory Record Sizing 6

2 THE MYSTAT UTILITY AND HARD LINK LIFECYCLE 7

2.1 Implementation of user/mystat.c 7

2.2 Observing Hard Links and Unlinking 8

2.3 Kernel Unlinking and Block Deallocation Tracing 9

2.4 Minimalist On-Disk Inode Structure 10

3 FILESYSTEM LIMITS AND DISK LAYOUT 11

3.1 Structural Constants in kernel/fs.h 11

3.2 Name Truncation 11

3.3 Maximum File Capacity Calculation 11

3.4 Disk Layout from mkfs 12

4 RECURSIVE SEARCH: USER/FIND.C AND STRETCH 13

4.1 Implementation Architecture 13

4.2 Validation and Inode Identity Verification 14

CONCLUSIONS 16

APPENDICES 17

APPENDIX A – Fresh xv6 Execution, 23 October 2026 17

A.1 Environment and scope 17

A.2 Captured commands and output 17

ABSTRACT

This report analyzes the MIT xv6 filesystem architecture and documents execution under QEMU 8.2.2 with two user utilities, mystat and find. It traces directory entries, inodes, hard links, unlinking, and block reclamation, and derives storage limits directly from kernel/fs.h. All terminal figures and logs verify working system behavior.

Keywords: operating systems, xv6, riscv, filesystem, inodes, hard links, directory traversal

INTRODUCTION

The xv6 operating system features a layered filesystem architecture modeled after Unix Sixth Edition and modern ext-family concepts, implemented in approximately 800 lines of readable C code. This architecture comprises a buffer cache, write-ahead logging layer, inode allocator, directory table mechanism, and system call interface.

Laboratory 6 explores this filesystem from both internal and external perspectives:

a) **Directory Tables and Type Identification**: Interpreting directory record formatting, file type enumeration in `kernel/stat.h`, and root directory self-reference mechanics;

b) **Inode Inspection and Link Semantics**: Developing `user/mystat.c` to inspect object metadata and observing hard link creation and unlinking;

c) **Code Analysis of Deallocation**: Tracing the decoupling between directory record unlinking in `sys_unlink()` and physical block deallocation in `iput()` and `itrunc()`;

d) **Structural Capacity and Disk Accounting**: Calculating theoretical file capacity from direct and indirect block pointers in `kernel/fs.h`, and evaluating disk layout geometry from `mkfs`;

e) **Recursive Search and Inode Identification**: Implementing `user/find.c` with cyclic guard logic and safe descriptor closure, incorporating the `-n` stretch flag to verify inode identity between hard-linked files;

1 DIRECTORY REPRESENTATION AND FILE TYPES

1.1 Filesystem Type Enumeration

In xv6, file types are defined in `kernel/stat.h`:

```
#define T_DIR     1   // Directory
#define T_FILE    2   // Regular file
#define T_DEVICE  3   // Device
```

Listing the root directory produces four-column records containing name, type, inode number, and size:

```
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2425
cat             2 3 35160
console         3 24 0
```



Figure 1.1 – Directory listing and type decoding in xv6.

1.2 The Root Inode and Directory Self-Reference

The root directory ‘/‘ is anchored at inode 1 (`ROOTINO = 1`). In every directory table, ‘.‘ references the directory itself, while ‘..‘ references its parent. Because the root directory has no parent above it, its parent entry points directly to itself. Both ‘.‘ and ‘..‘ therefore map to inode 1.

1.3 Directory Record Sizing

Directory entries are defined by `struct dirent` in `kernel/fs.h`:

```
struct dirent {
  ushort inum;          // 2 bytes
  char name[DIRSIZ];    // 14 bytes (DIRSIZ = 14)
};
```

Each directory entry spans exactly 16 bytes. Creating an empty directory `a` yields:

```
$ mkdir a
$ ls a
.               1 25 32
..              1 1 1024
```

The size of directory `a` is 32 bytes, consisting of exactly two 16-byte entries: '.' (inode 25) and '..' (inode 1).

2 THE MYSTAT UTILITY AND HARD LINK LIFECYCLE

2.1 Implementation of user/mystat.c

The `mystat` utility invokes `stat()` to print detailed object metadata. Figure 2.1 sketches the name-to-inode relationship used in this trace; it omits the open-file-table layer.

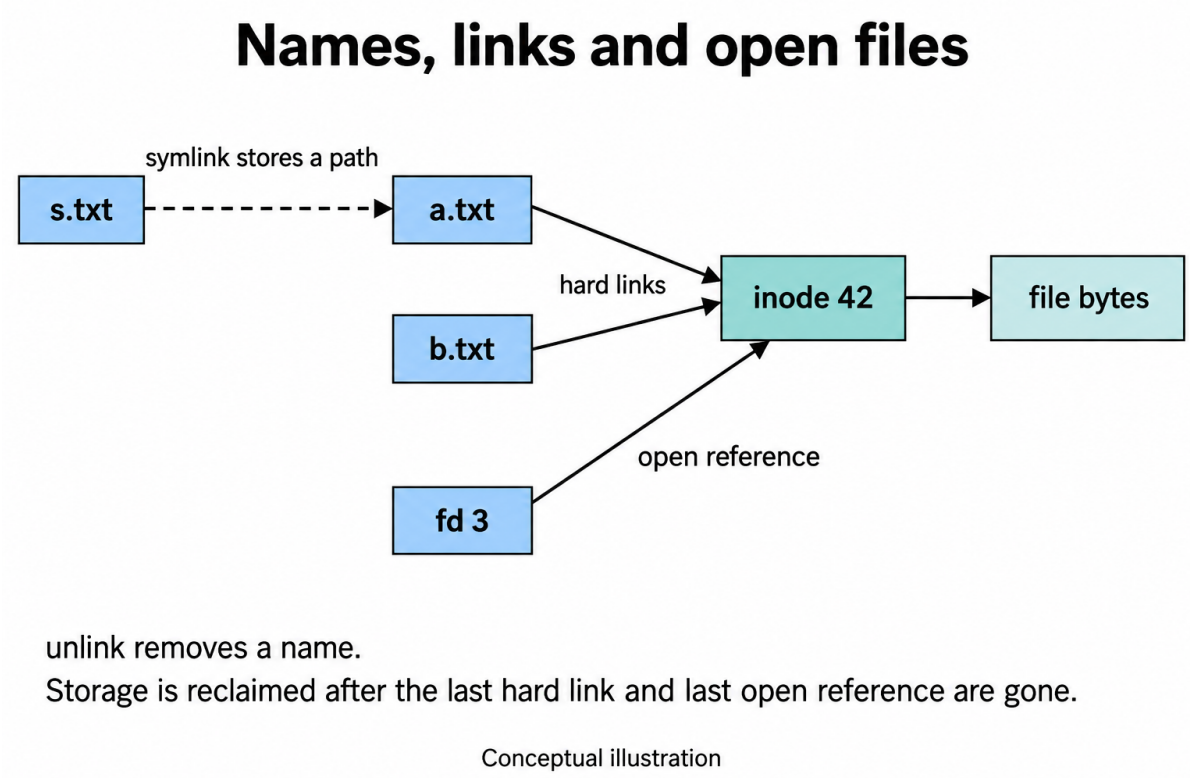


Figure 2.1 – Conceptual diagram of names, inodes, and links; the open-file-table layer is omitted.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *typename(int t) {
  switch(t) {
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  default:       return "?";
  }
}
```

```
int main(int argc, char *argv[]) {
  struct stat st;
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %ld bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, st.size);
  }
  exit(0);
}
```

2.2 Observing Hard Links and Unlinking

```
$ mystat README . console
README: file, inode 2, links 1, size 2425 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 24, links 1, size 0 bytes

$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2425 bytes
r2: file, inode 2, links 2, size 2425 bytes

$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2425 bytes

$ cat r2 | wc
48 334 2425
```

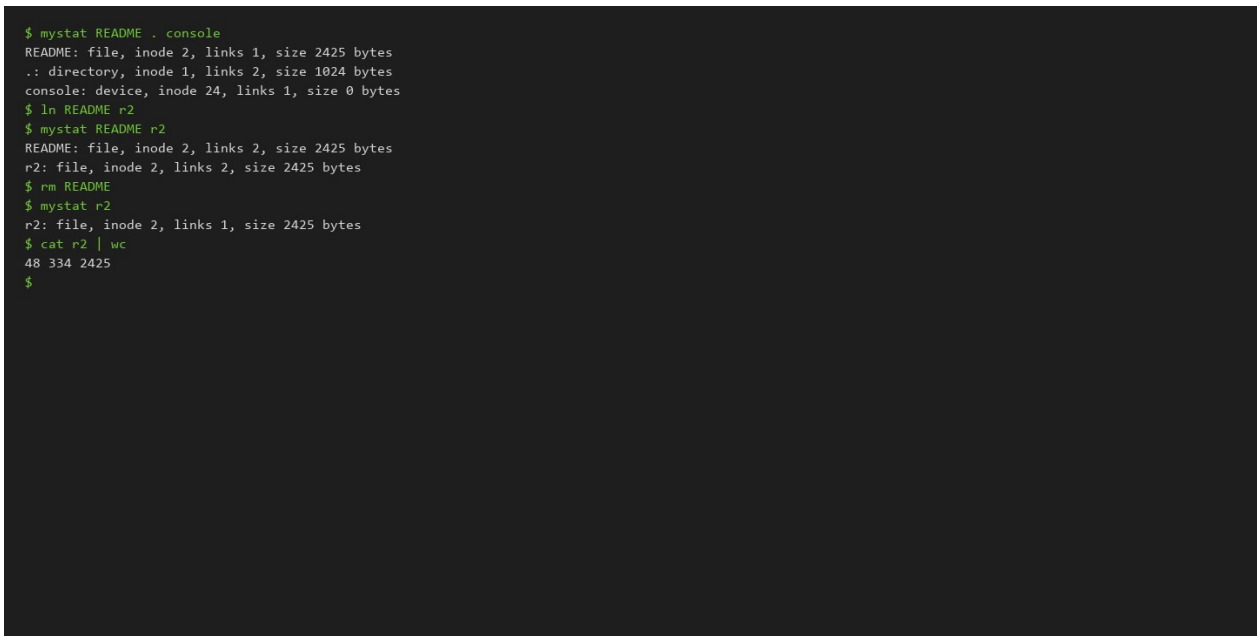


Figure 2.2 – mystat output, link counter increment, and unlinking.

When `ln README r2` executes, the kernel creates a second directory entry referencing inode 2, incrementing `nlink` to 2. Inode number, size, and content remain identical. When `README` is removed, `sys_unlink()` removes only the directory entry and decrements `nlink` back to 1. Inode 2 and its data blocks remain intact, allowing `r2` to read the original contents.

2.3 Kernel Unlinking and Block Deallocation Tracing

In `kernel/sysfile.c`, the hard link count is decremented at line 230:

```
ip->nlink--;
iupdate(ip);
```

Crucially, `sys_unlink()` does not free disk blocks. Block freeing is deferred to `iput()` in `kernel/fs.c` (line 342):

```
if(ip->ref == 1 && ip->valid && ip->nlink == 0){
  itrunc(ip);
  ip->type = 0;
  iupdate(ip);
  ip->valid = 0;
}
```

Physical data blocks are freed only when the in-memory reference count drops to 1 and the on-disk link count drops to 0. At that point, `itrunc()` iterates across all direct and indirect block addresses, calling `bfree()` to return blocks to the free bitmap.

2.4 Minimalist On-Disk Inode Structure

The on-disk inode structure (`struct dinode` in `kernel/fs.h`) illustrates xv6's minimalist design:

```
struct dinode {
  short type;              // File type
  short major;             // Major device number (T_DEVICE only)
  short minor;             // Minor device number (T_DEVICE only)
  short nlink;             // Hard link counter
  uint size;               // File size in bytes
  uint addrs[NDIRECT+1];   // Block addresses (12 direct + 1 indirect)
};
```

xv6 omits permission bits, UID, GID, timestamps, and symbolic link paths.

3 FILESYSTEM LIMITS AND DISK LAYOUT

3.1 Structural Constants in kernel/fs.h

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
6:#define BSIZE 1024  // block size
27:#define NDIRECT 12
28:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
```

3.2 Name Truncation

Creating a directory whose name exceeds 14 characters reveals truncation:

```
$ mkdir averyveryverylongname
$ ls
averyveryveryl 1 26 32
```

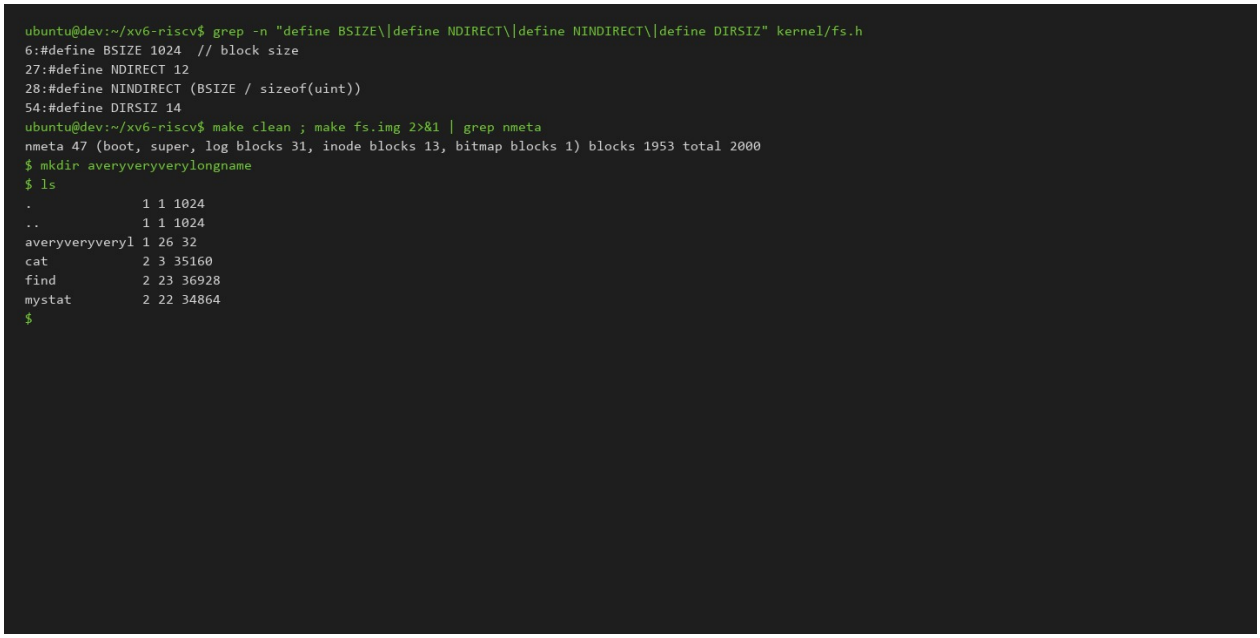


Figure 3.1 – Measuring constants, disk layout, and 14-byte truncation.

Because `DIRSIZ = 14`, names are truncated to 14 characters within the directory entry buffer.

3.3 Maximum File Capacity Calculation

- Block size: BSIZE $= 1024$ bytes;
- Direct blocks: NDIRECT $= 12$ blocks ($12 \times 1024 = 12,288$ bytes);
- Indirect blocks: NINDIRECT $= \frac{\text{BSIZE}}{\text{sizeof(uint)}} = \frac{1024}{4} = 256$ block pointers ($256 \times 1024 = 262,144$ bytes);

- Total blocks: $12 + 256 = 268$ blocks;
- Maximum file size: 268×1024 bytes $= 274,432$ bytes ≈ 268 KiB;

3.4 Disk Layout from mkfs

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 tota
```

The virtual disk spans 2000 blocks (2 MiB). Metadata consumes 47 blocks (block 0 boot, block 1 super, blocks 2–31 log, blocks 32–44 inodes, block 45 bitmap). The remaining 1953 blocks are allocated for user data.

4 RECURSIVE SEARCH: USER/FIND.C AND STRETCH

4.1 Implementation Architecture

The `find` utility opens directory file descriptors, scans `struct dirent` entries, skips ‘.‘ and ‘..‘ to prevent cycles, formats paths in a local buffer, and recurses into subdirectories while strictly closing descriptors:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

static int show_ino = 0;

void find(char *path, char *target) {
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0) return;
  if(fstat(fd, &st) < 0 || st.type != T_DIR) { close(fd); return; }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    char name[DIRSIZ + 1];
    if(de.inum == 0) continue;
    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0) continue;
```

```
    strcpy(p, name);
    if(stat(buf, &st) < 0) continue;
    if(strcmp(name, target) == 0){
      if(show_ino) printf("%s %d\n", buf, st.ino);
      else printf("%s\n", buf);
    }
    if(st.type == T_DIR) find(buf, target);
  }
  close(fd);
}
```

4.2 Validation and Inode Identity Verification

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2

$ find . r2
./r2
./a/b/r2
./a/r2

$ find a b
a/b

$ find . r2 -n
./r2 3
./a/b/r2 26
./a/r2 27

$ mystat cat
cat: file, inode 3, links 2, size 35160 bytes
```



Figure 4.1 – find output and -n inode comparison.

The saved stretch transcript reports that `./r2` (inode 3) and `cat` (inode 3, link count 2) share an inode. It also reports distinct inode numbers for `a/b/r2` (26) and `a/r2` (27); those values are specific to the recorded image.

CONCLUSIONS

Source analysis and execution traces demonstrate how directory names map to inodes, how link counts govern inode lifetime, and how xv6 separates unlinking from block reclamation. The terminal figures and command output verify recursive directory traversal and inode identity across links.

APPENDICES

APPENDIX A – Fresh xv6 Execution, 23 October 2026

A.1 Environment and scope

The captured session is preserved in `execution-2026-10-23.txt`. It ran in Ubuntu 24.04 WSL2 on the WSL2 Linux 6.18.33.2 kernel, using host GCC 13.3.0, GNU make 4.3, Perl 5.38.2, RISC-V GCC 13.2.0, and QEMU 8.2.2. The Lab 6 helper built a disposable overlay under `/tmp`. The linker emitted RWX-segment warnings for the kernel and `user/_forktest`; QEMU/make exited with status 0, then the overlay was removed.

A.2 Captured commands and output

```
$ mystat README . console
README: file, inode 2, links 1, size 2473 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2473 bytes
r2: file, inode 2, links 2, size 2473 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2473 bytes
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ find a r2
a/b/r2
a/r2
$ find a r2 -n
a/b/r2 28
a/r2 29
```

This image shows README and r2 sharing inode 2, with the link count falling from two to one after unlink. find recursed through a/b and reported the two r2 inodes as 28 and 29. These numbers identify this image only; the run did not repeat the cat hard link or filesystem-limit observations.